COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithm	**SECTION:**	CPE12S4
DATE PERFORMED:	Oct 21, 2026	**DATE SUBMITTED:**	Oct 21, 2024
NAME:	Cardenas and De Vera	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY 14.1 ALGORITHMIC COMPLEXITY

1. PROCEDURE OUTPUT

What procedures did you perform?

Tabulated below, are a number of functions against n (from 1 to 10)

```
A = (log2 n) {log to base 2 of n}
B = n {linear in n}
C = (* n (log2 n)) {n log n}
D = (* n n) {quadratic in n}
E = (* n n n) {cubic in n}
F = (expt 2 n) {exponential in n}
G = (expt 3 n) {exponential in n}
H = (fact n) {factorial in n}
```

n	A	B	C	D	E	F	G	H
1	0.0	1	0.0	1	1	2	3	1
2	1.0	2	2.0	4	8	4	9	2
3	1.0	3	4.0	9	27	8	27	6
4	2.0	4	8.0	16	64	16	81	24
5	2.0	5	11.0	25	125	32	243	120
6	2.0	6	15.0	36	216	64	729	720
7	2.0	7	19.0	49	343	128	2187	5040
8	3.0	8	24.0	64	512	256	6561	40320
9	3.0	9	28.0	81	729	512	19683	362880
10	3.0	10	33.0	100	1000	1024	59049	3628800

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x".

n	F	G	H
10	10^-10	10^-8	10^-6
20	10^-7	10^-3	10^4
30	10^-4	10^0	10^18
40	10^-1	10^5	10^34
50	10^1	10^10	10^50
60	10^4	10^15	10^68
70	10^7	10^19	10^86
80	10^10	10^24	10^105
90	10^13	10^29	10^124
100	10^16	10^34	10^144

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take

10,000,000,000,000,000,000,000,000,000,000,000 years

approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, ie 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say, that from a computational perspective, the universe is a small thing.

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

	Best Case	Worst Case	Analysis
Algorithm 1: Bubble Sort	O(n)	$O(n^2)$	It repeatedly compares adjacent elements and swaps them if they are in the wrong order. In the best case, O(n) it will check the array in a single pass and on the other hand in the worst case O(n^2) this will be a quadratic time growth to make it inefficient for the large input. For example n= 10,000
Algorithm 2: Merge Sort	O(n log n)	O(n log n)	It uses a Divide and Conquer approach and it will be recursively splitting the data into halves and merging them back in sorted order. It maintains

			a steady log-linear. Best case and worst case are the same because it is highly scalable and efficient for large datasets.

Table 14-1. Best-and Worst-Case Analysis using Theoretical Tools

	ALGORITHM 1	ALGORITHM 2
BEST CASE	In experimental analysis and implementation, bubble sort can have its input in an already sorted order–either ascending and descending order. As we all know, the algorithm compares adjacent elements and checks whether they need to be switched or not. Testing its time complexity in different population size elements, the execution runtime is ***linear*** to the input size. Hence, bubble sort is ***O(n)***.	Merge Sort on the other hand, who uses the divide-and-conquer strategy, the data is being divided into subsets. With this, during its execution time and experimental analysis, merge sort can actually run efficiently on larger datasets and population sizes. This is because its execution time grows at a slower rate. Merge sort falls under the ***O(n log n)*** time complexity.
WORST CASE	In its worst case scenario, when the bubble sort is being tested especially in a large population size, having a linear time complexity can have a long execution time. Therefore, to compensate for this, the $O(n^2)$ time complexity is used to make execution time slower.	Here on the merge sort, data requires a full sequence of divide and conquer strategy no matter how they are arranged. But, since its execution grows gradually, the actual performance depends on how the computer performs.

Input size (n)	Bubble Sort Time	Merge Sort Time
100	Measure	Measure
1,000	Measure	Measure
10,000	Measure	Measure

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PSEUDO CODE:

```
ALGORITHM LinearSearch(A, n, target)
   FOR i = 0 TO n - 1
      IF A[i] == target
         RETURN i
   RETURN -1
ALGORITHM BinarySearch(A, n, target)
   low = 0
   high = n - 1
   WHILE low <= high
      mid = low + (high - low) / 2

      IF A[mid] == target
         RETURN mid
      ELSE IF A[mid] < target
         low = mid + 1
      ELSE
         high = mid - 1
   RETURN -1
```

- Best Case: target is the middle element and requires only one comparison.
- Worst Case: Search continues by halving the range until the target is found or the range is empty.
- Space Usage: Iterative implementation uses a constant amount of additional memory.

Time and Space Complexities

Analysis	Linear Search	Binary Search
Best-case Time	O(1)	O(1)
Average-case Time	O(n)	O(log n)
Worst-case Time	O(n)	O(log n)
Auxiliary Space	O(1)	O(1)
Requirement	Sorted data not required	Data must be sorted

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1000	0 ms	7 ms
10,000	0 ms	68 ms
100,000	0 ms	701 ms

Analysis

Linear Search is simple to implement and works with sorted or unsorted data, but its execution time increases as the number of elements grows. Binary Search is more efficient for large datasets because it eliminates half of the remaining search range after each comparison. However, it requires sorted data to work correctly. The theoretical analysis predicts linear growth for Linear Search and logarithmic growth for Binary Search in the worst case. The empirical analysis can demonstrate these differences by measuring their actual execution times on the same computer and dataset.

Conclusions

Cardenas:

In this activity, I learned about time and space complexity and how they affect the performance of an algorithm. I learned that Linear Search checks the elements one by one, while Binary Search divides the search range in half to find the target faster. I also learned that Binary Search works only with sorted data. Through this activity, I understood that choosing the right algorithm is important, especially when working with large amounts of data.

De Vera:

This Hands-On Activity is designed for us to learn and grasp the concepts of the algorithms and their time and space complexity. Through this theoretical and experimental analysis, I was able to compare algorithms based on their efficiency and performance. I was also able to identify the importance of choosing the appropriate algorithm techniques especially when working on different population size to optimize the computer's performance and execution time.

AI DISCLOSURE

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.